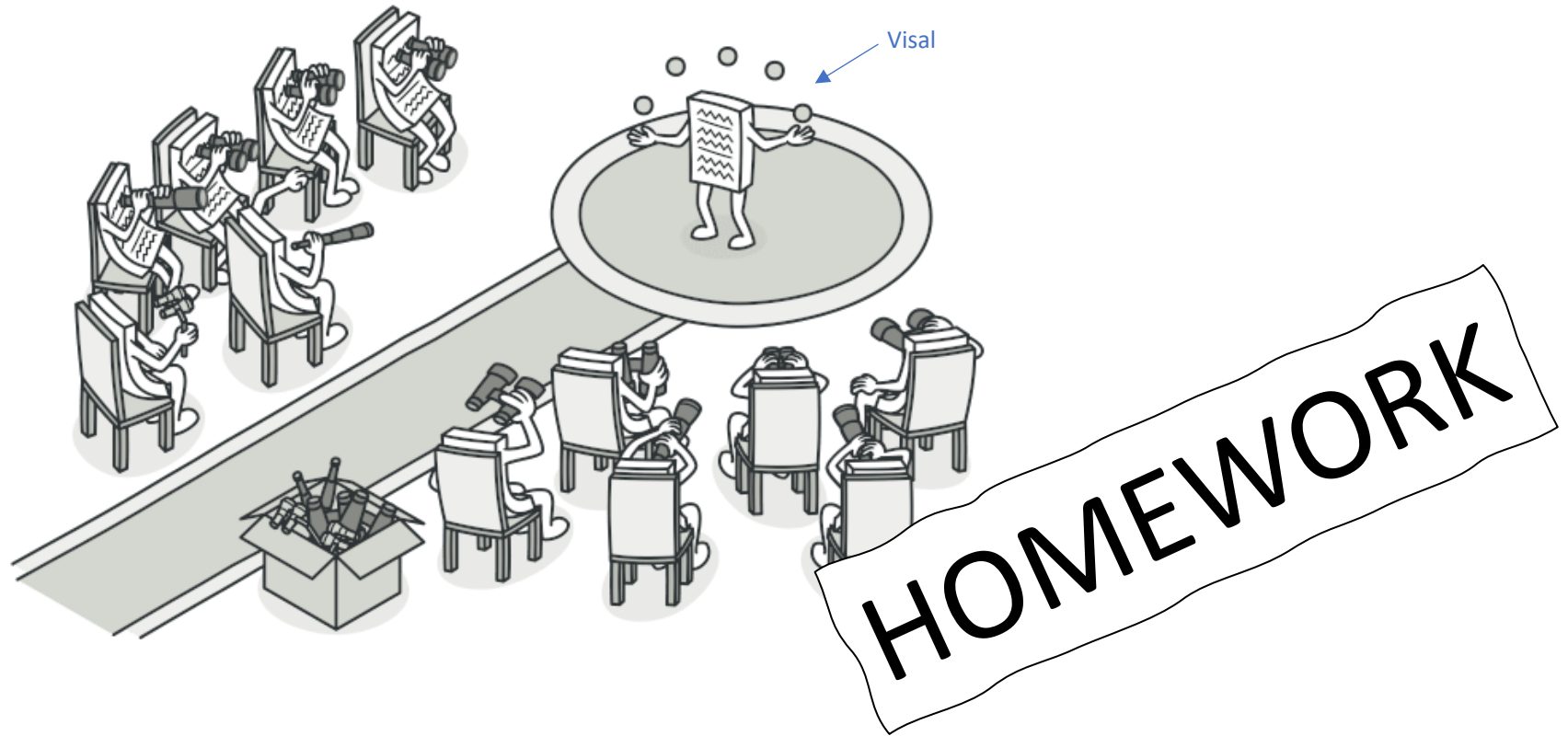


MOBILE DEVELOPMENT 2

W5 - Observer Pattern, State Management





How to submit?



- ✓ Create a dedicated git-hub repository **W5 - Observer Pattern, State Management**
- ✓ Provide each exercise on a separate FOLDER :

`lib/EXERCISE-1`

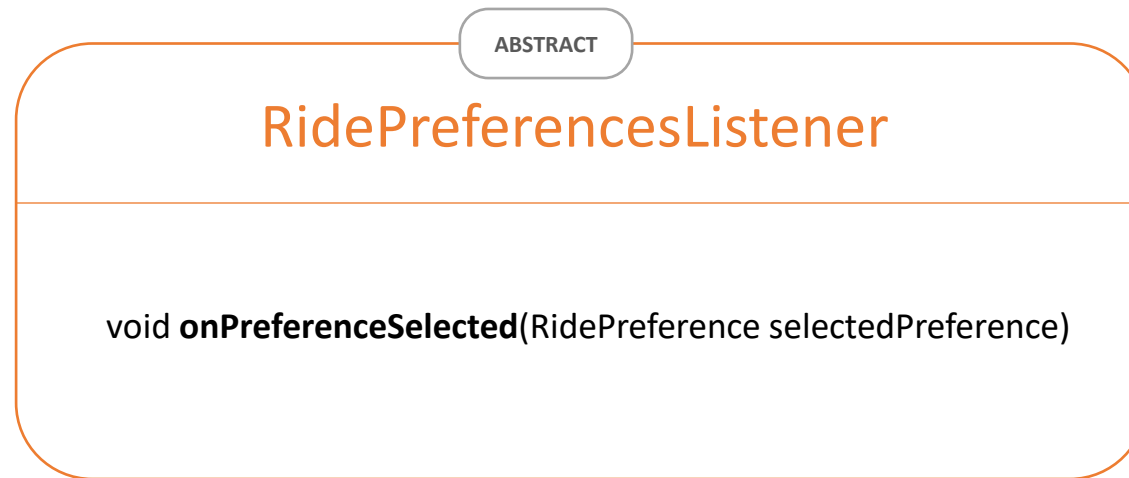
`lib/EXERCISE-2`

- ✓ Provide your GitHub link on MS Team

Implement an observer Pattern

Implement a Listener for the **RidePreferencesService**

We want a console logger to get notified when the current selected preference has changed

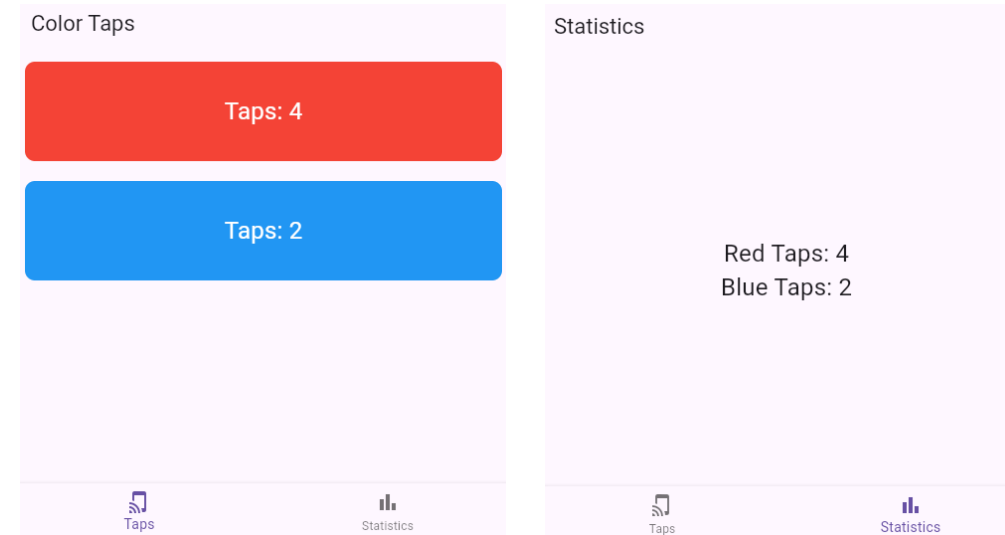


1. Create the **abstract listener** **RidePreferencesListener**
2. Manage a list of listeners in the **RidePreferencesService** and code the `notifyListeners()`, `addListener()`...
3. Implement a **ConsoleLogger** that implements the **RidePreferencesListener**
 - *On preference changed : log a message on console !*
4. Test the service and the **ConsoleLogger** on a dedicated `main()`

Refactor the **ColorTaps** App with a **Provider**

Start from the code provided during the course (color tap activity)

1. Create a **model** (ColorCounters) to handle the 2 counters
2. Provide the model to the app root widget using a **ChangeNotifierProvider**
3. Remove all state drilling and states in every widgets
4. Wrap the widget who needs the model with a consumer **Consumer<ColorCounters>**
5. Use the **colorCounters** to get or increment the colors



How **efficient** is the new approach ?

Let's analyze what is refreshed in the new and the old approaches

- ✓ Add **some print statements** in each build() methods
 - Regarding the **previous approach (provided code)**
 - Regarding the new approach (your code)
- ✓ Analyze the **number of widgets** rebuild on each approach
- ✓ Be ready to **explain your analysis** during the next session

```
@override
Widget build(BuildContext context) {
  // ignore: avoid_print
  print("Home rebuilt");
}
```

Log each widget build() action on console

